

Serving Metrics — Why Average Latency Lies, Worked Out by Hand

TTFT, TPOT, PERCENTILES, AND LITTLE'S LAW — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It defines the handful of numbers that actually decide whether an LLM deployment is healthy — and shows, with arithmetic, why the most natural number to report (the *mean* latency) is the one most likely to lie to you. We use the *Nexus* fraud-detection baseline throughout: a 70B model whose v1 deployment posts a 295 ms time-to-first-token and a 22 ms per-token decode rate. Every figure is produced by a small reference script (Appendix A, `m01_t03_engine.py`) so the numbers are exact and reproducible.

The one idea. LLM latency is a *distribution*, not a number, and it is heavily right-skewed: most requests are fast, a few are catastrophic. A mean blends the two phases of inference and hides the tail. You must report *percentiles* — p50, p90, p99, p99.9 — because the tail is what your users feel.

CONCEPTS COVERED, IN ORDER

TTFT and TPOT; end-to-end latency = $\text{TTFT} + N \cdot \text{TPOT}$; throughput = batch/step time; the latency distribution and its percentiles p50/p90/p99/p99.9; a worked example where the mean sits below p99 by $\sim 10\times$; why the mean misleads on skewed data; and Little's Law $L = \lambda W$ relating concurrency, arrival rate, and latency.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The four numbers that matter	2
2	Step 1 — End-to-end latency = $\text{TTFT} + N \cdot \text{TPOT}$	2
3	Step 2 — Throughput = batch / step time	3
4	Step 3 — Latency is a distribution, not a number	3
5	Step 4 — A worked example where the mean lies	4
6	Step 5 — Little's Law: concurrency = arrival rate $\times$ latency	5
7	The argument, at a glance	6
7.1	Equation cheat-sheet	6
	Appendix A — Reference implementation	6

1 The four numbers that matter

“The average response time is 120 ms” is one of the most dangerous sentences in LLM serving. It hides four metrics — and averages across two phases (prefill and decode) that have no business being averaged together.

Metric	Definition	Lever
TTFT	Time to First Token: arrival $\rightarrow$ first output token on the wire.	prefill speed, queueing
TPOT	Time Per Output Token: mean gap between successive decode tokens.	decode speed (bandwidth)
Throughput	Output tokens/sec across all requests on the cluster.	batching
Goodput	Throughput of requests that <i>met their SLO</i> .	scheduling, admission

Intuition

TTFT and TPOT measure two physically different machines. TTFT is dominated by *prefill* — a big compute-bound matmul over the whole prompt — plus any time spent waiting in the scheduler queue. TPOT is the *decode* cadence: one memory-bandwidth-bound weight-read per token. Averaging them into a single “latency” number is like averaging a sprinter’s start reaction with their stride rate: the blend describes neither.

2 Step 1 — End-to-end latency = TTFT + $N \cdot \text{TPOT}$

A request finishes when its last token is emitted. So total wall-clock is the time to the first token plus one TPOT for each remaining token.

End-to-end request latency

$$L_{e2e} = \text{TTFT} + N_{\text{out}} \cdot \text{TPOT}.$$

Nexus v1, plugged in

TTFT itself decomposes into queue wait plus prefill of the 6200-token prompt:

$$\text{TTFT} = \underbrace{25 \text{ ms}}_{\text{queue}} + \underbrace{275 \text{ ms}}_{\text{prefill}} = 295 \text{ ms}.$$

$$\begin{aligned} L_{e2e} &= \text{TTFT} + N_{\text{out}} \cdot \text{TPOT} = 295 \text{ ms} + 180 \times 22 \text{ ms} \\ &= 295 \text{ ms} + 3960 \text{ ms} = 4255 \text{ ms} \approx 4.25 \text{ s}. \quad \checkmark \end{aligned}$$

Decode’s share: $3960/4255 = \mathbf{93\%}$ of the wall-clock is the memory-bound decode loop.

Which metric to attack is a function of the workload

For Nexus (6200-token prompt, 180-token answer) decode is 93% of latency, so **TPOT is the target** — quantization and speculative decoding. Flip the shape — a 30-token prompt with a 500-token answer — and decode dominates even harder; flip it the other way — a 10-token prompt, 5-token answer for code completion — and **TTFT is everything**.

Always profile the real shape before optimizing.

3 Step 2 — Throughput = batch / step time

Latency is per request; *throughput* is per GPU. One decode step reads the weights once and advances every request in the batch by one token, so:

Decode throughput

$$\text{throughput} = \frac{\text{batch size}}{\text{step time}} \quad [\text{tokens/s}].$$

Batching multiplies throughput, not latency

With a 22 ms step (one weight read, shared by the whole batch):

batch	step time	throughput
1	22 ms	$1/0.022 \approx 45 \text{ tok/s}$
32	22 ms	$32/0.022 \approx 1455 \text{ tok/s}$
64	22 ms	$64/0.022 \approx 2909 \text{ tok/s}$

The step time barely moves with batch (decode is bandwidth-bound on the *same* weight read), so throughput scales almost linearly with batch — while each individual user still waits one full 22 ms step per token.

Watch out

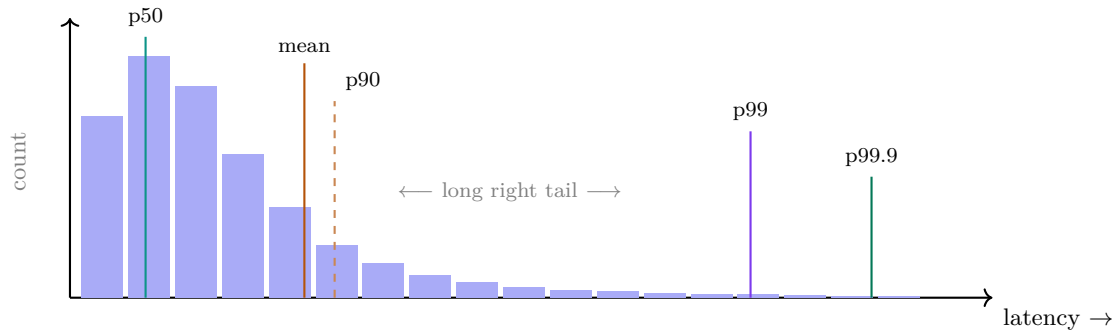
Throughput and latency are *different objectives*. Batching 32× the requests gives ~32× the tokens/sec but does *not* make any single user’s stream faster. Reporting a throughput win as if it were a latency win is the most common sleight of hand in capacity planning.

4 Step 3 — Latency is a distribution, not a number

Run the same request a thousand times and you do not get one latency — you get a *distribution*. For LLMs it is heavily **right-skewed**: a dense cluster of fast responses and a long tail of slow ones (preemption, KV-cache pressure, queue spikes, gateway retries). A single summary number has to choose what to emphasize, and the *mean* chooses badly.

Intuition

The **percentile** p_q is the latency below which $q\%$ of requests fall. p50 (the *median*) is the typical experience; p99 is the worst 1%; p99.9 is the worst 1 in 1000. At Nexus’s target of 10,000 req/s, the “p99” is not a rare curiosity — it is 100 requests *every second* sitting above the line. The tail is a population, not an anecdote.



The mean is dragged *rightward* by the tail until it sits to the right of most of the data — yet still far to the *left* of p99. It describes nobody: not the typical user (p50), not the suffering user (p99). The next step makes that concrete with numbers.

5 Step 4 — A worked example where the mean lies

Take 20 observed request latencies (in ms): eighteen healthy, two tail events.

{40, 42, 45, 47, 48, 50, 51, 52, 55, 58, 60, 62, 65, 70, 75, 80, 90, 110, **1900, 3200**}.

Compute the summary statistics

$$\begin{aligned} \text{mean} &= \frac{1}{20} \sum x_i = \frac{6200}{20} = 310 \text{ ms}, \\ \text{p50 (median)} &= 59 \text{ ms} \quad (\text{average of the 10th, 11th values}), \\ \text{p90} &= 289 \text{ ms}, \quad \text{p99} = 2953 \text{ ms}, \quad \text{p99.9} = 3175 \text{ ms}. \end{aligned}$$

Statistic	Value	Reading
mean	310 ms	dragged up by two outliers
p50	59 ms	the <i>typical</i> request
p90	289 ms	still under the mean
p99	2953 ms	the suffering 1%
p99.9	3175 ms	near the worst case

The mean describes a request that does not exist

The mean (310 ms) is 5.3× **the median** (59 ms) and yet only 1/10 of **p99** (2953 ms). *Ninety percent* of requests are faster than the mean — so quoting it makes the service sound an order of magnitude worse than the typical user feels, while simultaneously hiding the 50× tail that the worst users actually hit. The mean is a number that describes no real request. Report {p50, p90, p99, p99.9} instead.

Mini-example — the p99.9/p99 ratio is a health signal

A healthy tail has $\text{p99.9}/\text{p99} \lesssim 2.5$. Here it is $3175/2953 \approx 1.1$ — the two outliers are similar in magnitude, so the tail is “shaped” but not pathological. A ratio of 70× (e.g. p99 = 250 ms but p99.9 = 18,000 ms) signals something catastrophic on rare events: preemption, KV-pool exhaustion, or a gateway retrying indefinitely. None of that is visible on a p99 dashboard — which is exactly why you also track p99.9.

Watch out

Percentiles do *not* average. You cannot take the mean of per-server p99s to get a fleet p99 — the fleet tail can be far worse than any single server's. Aggregate the raw latency samples (or use a mergeable sketch like t-digest/HDR histogram), then take the percentile of the merged set.

6 Step 5 — Little's Law: concurrency = arrival rate $\times$ latency

One more number ties latency to load. **Little's Law** is an exact, distribution-free identity for any stable system: the average number of requests *in flight* equals the arrival rate times the average time each spends in the system.

Little's Law

$$L = \lambda W \quad \left(\text{concurrency} = \text{arrival rate} \times \text{latency} \right).$$

How many requests is Nexus juggling?

With $W = L_{e2e} = 4.25 \text{ s}$ (Step 1):

λ (req/s)	$L = \lambda W$	<i>meaning</i>
5	$5 \times 4.25 = 21.3$	≈ 21 requests in flight
6	$6 \times 4.25 = 25.5$	≈ 26 in flight
10	$10 \times 4.25 = 42.5$	≈ 43 in flight

Nexus v1 can hold only ~ 25 concurrent requests before its KV-cache pool is exhausted. Solving for the sustainable arrival rate:

$$\lambda_{\max} = \frac{L_{\text{cap}}}{W} = \frac{25}{4.25 \text{ s}} \approx 5.9 \text{ req/s.} \quad \checkmark$$

Latency and capacity are the same problem

Little's Law makes the link unavoidable: at $W = 4.25 \text{ s}$, serving the SLO target of 10,000 req/s would require $L = \lambda W = 42,550$ concurrent slots. You get there two ways — and only two: raise concurrency L (bigger batches, more KV capacity: Modules 04–05) or cut latency W (faster decode: Module 02). Cutting W helps *twice*, because lower latency means each slot frees up sooner. This is why latency work and throughput work are not separate projects.

Intuition

Little's Law is just bookkeeping on a queue: if 6 customers walk in per second and each stays 4.25 seconds, then at any frozen instant there are $6 \times 4.25 \approx 26$ customers inside. It assumes nothing about the latency distribution — skew, tails, batching all wash out in the long-run average. That robustness is why it is the first equation every capacity plan is built on.

7 The argument, at a glance

#	Step	Result
1	E2E latency	$\text{TTFT} + N \cdot \text{TPOT} = 295 + 180 \cdot 22 = 4255 \text{ ms}$; decode 93%
2	Throughput	batch/step; $64/22 \text{ ms} \approx 2909 \text{ tok/s}$ (latency unchanged)
3	Distribution	latency is right-skewed; report percentiles
4	Mean lies	mean 310 = $5.3 \times p50$, = $0.1 \times p99$; 90% beat the mean
5	Little's Law	$L = \lambda W$; cap 25 $\Rightarrow \lambda_{\max} \approx 5.9 \text{ req/s}$

7.1 Equation cheat-sheet

End-to-end latency	$L_{\text{e2e}} = \text{TTFT} + N_{\text{out}} \cdot \text{TPOT}$
TTFT decomposition	$\text{TTFT} = t_{\text{queue}} + t_{\text{prefill}}$
Throughput	batch/step time
Percentile p_q	latency below which $q\%$ of requests fall
Tail health	$p99.9/p99 \lesssim 2.5$ (healthy)
Little's Law	$L = \lambda W$
Max arrival rate	$\lambda_{\max} = L_{\text{cap}}/W$

Appendix A — Reference implementation

Every number above is produced by `m01_t03_engine.py` (provided alongside this PDF). It computes the end-to-end latency, the throughput table, the percentiles of the 20-sample distribution, and the Little's-Law concurrency figures.

```

1  """
2  Reference implementation for: Module 01 - Topic 3
3  "Serving Metrics - why average latency lies."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Uses the Nexus v1 baseline numbers from Module 01:
9      TTFT p99 = 295 ms (queue ~25 ms + prefill ~275 ms over 6200 tokens)
10     TPOT    = 22 ms/token, N_out = 180 output tokens
11     end-to-end p99 = TTFT + N_out * TPOT
12  """
13  import statistics
14
15  # -----
16  # 0. The Nexus v1 serving baseline.
17  # -----
18  TTFT_ms    = 295.0      # time to first token (queue + prefill), p99
19  TPOT_ms    = 22.0       # time per output token, decode
20  N_out       = 180        # output tokens
21  queue_ms   = 25.0       # scheduler queue wait
22  prefill_ms = 275.0      # prefill of 6200 prompt tokens
23
24  # -----
25  # 1. End-to-end latency = TTFT + N * TPOT.
26  # -----
27  e2e_ms = TTFT_ms + N_out * TPOT_ms
28  print("== 1. End-to-end latency ==")
29  print(f"TTFT = queue {queue_ms:.0f} + prefill {prefill_ms:.0f} = {queue_ms+prefill_ms:.0f} ms")

```

```

30 print(f"E2E = TTFT + N*TPOT = {TTFT_ms:.0f} + {N_out}*{TPOT_ms:.0f} = {e2e_ms:.0f} ms = {e2e_ms/1000:.2f} s")
31 print(f"decode share = N*TPOT / E2E = {100*N_out*TPOT_ms/e2e_ms:.1f}% (memory-bound regime)")
32
33 # -----
34 # 2. Throughput = batch / step_time. One decode step serves the whole batch.
35 # -----
36 print("\n== 2. Throughput = batch / step_time ==")
37 for batch in [1, 32, 64]:
38     step_s = TPOT_ms / 1000.0          # one decode step, ~bandwidth-bound, batch-independent
39     tput = batch / step_s              # output tokens/sec across the batch
40     print(f"  batch={batch:3d}: step={TPOT_ms:.0f} ms -> throughput = {batch}/{step_s:.3f}s = {tput:7.0f} tok/s")
41 print(" (Nexus v1 measured ~1,100 tok/s; latency per user is unchanged by batching.)")
42
43 # -----
44 # 3. Why the mean lies: a small right-skewed latency sample.
45 #   Most requests are fast; a few tail events dominate the percentiles.
46 # -----
47 # 20 request latencies in ms: 18 healthy, 2 tail (preemption / KV pressure).
48 lat = [40, 42, 45, 47, 48, 50, 51, 52, 55, 58,
49        60, 62, 65, 70, 75, 80, 90, 110, 1900, 3200]
50
51 def pct(xs, p):
52     xs = sorted(xs)
53     k = (len(xs) - 1) * (p / 100.0)
54     lo = int(k); hi = min(lo + 1, len(xs) - 1)
55     return xs[lo] + (xs[hi] - xs[lo]) * (k - lo)
56
57 mean = statistics.mean(lat)
58 print("\n== 3. Why the mean lies (20-request sample, ms) ==")
59 print(f"  n={len(lat)}  min={min(lat)}  max={max(lat)}")
60 print(f"  mean = {mean:.1f} ms")
61 print(f"  p50  = {pct(lat,50):.0f} ms")
62 print(f"  p90  = {pct(lat,90):.0f} ms")
63 print(f"  p99  = {pct(lat,99):.0f} ms")
64 print(f"  p99.9= {pct(lat,99.9):.0f} ms")
65 print(f"  mean({mean:.0f}) sits ABOVE p90({pct(lat,90):.0f}) but far below p99({pct(lat,99):.0f}): "
66       f"{100*sum(x<mean for x in lat)/len(lat):.0f}% of requests are FASTER than the mean")
67 print(f"  p99/p50 tail ratio = {pct(lat,99)/pct(lat,50):.1f}x ; p99.9/p99 = {pct(lat,99.9)/pct(lat,99):.1f}x")
68
69 # -----
70 # 4. Little's Law: L = lambda * W (concurrency = arrival rate x latency).
71 # -----
72 print("\n== 4. Little's Law  L = lambda * W ==")
73 W = e2e_ms / 1000.0          # mean time in system, seconds
74 for lam in [5, 6, 10]:      # arrivals per second
75     L = lam * W
76     print(f"  lambda={lam:2d} req/s, W={W:.2f} s -> L = {L:.1f} concurrent requests in flight")
77 # Inverse: given a concurrency cap, what arrival rate can we sustain?
78 L_cap = 25                  # Nexus v1 ~25 concurrent before KV pool exhaustion
79 lam_max = L_cap / W
80 print(f"  cap L={L_cap} concurrent, W={W:.2f}s -> max lambda = L/W = {lam_max:.1f} req/s sustainable")
81 print(f"  to reach 10,000 req/s at W={W:.2f}s needs L = {10000*W:.0f} concurrent slots (Modules 4-5).")

```